

Deltazium

Debezium + Kafka 기반 Oracle CDC 파이프라인 · 복구(replay) 아키텍처 · 웹 제어 콘솔

최남희 · 단독 설계·개발 · 2026.07 · github.com/namhee0812/Deltazium · 상용 CDC 솔루션(DeltaStream) 개발 경험 기반의 검증 프로젝트

1. 프로젝트 개요

Oracle의 변경(CDC)을 LogMiner로 캡처해 **같은 스트림을 두 갈래로 병행 적재**한다 — 타깃 Oracle에는 현재 상태를(실적재), Iceberg/S3에는 모든 변경 이력을(append-only changelog). changelog는 미러링이 아니라 **특정 SCN 시점부터의 복구(replay)**를 위한 원본이며, 이 복구 아키텍처의 성립을 실제 배선으로 증명하는 것이 프로젝트의 목적이다. 그 위에 테이블 등록·사전 점검·DDL 승인·복구 트리거를 담당하는 제어면(Spring Boot + React)을 얹었다.

72

자동화 테스트

8.4k

코드 라인 (Java 4.8k · TS 3.7k)

34k+

실트래픽 E2E 검증 (행)

0

캡처 장애 시 유실 (건)

2. 아키텍처

Oracle(SRC) —Debezium source(LogMiner)—> Kafka(KRaft) —> Debezium JDBC sink —> Oracle(TGT) · 실적재(현재)
└─ Iceberg sink (append) —> Iceberg / MinIO(S3)

복구: recovery-job —(Iceberg scan · SCN 순서 복원 · envelope 재조립)—> 복구 토픽 —> recovery-sink(동일 apply)

[Web UI(React)] ≠ [backend(Spring Boot): 등록·사전 점검 · DDL 승인 · 복구 트리거 · 이벤트 이력 · 실측 메트릭]

컴포넌트	역할	보존/특성
Kafka (KRaft)	짧은 버퍼 · fan-out (두 sink가 독립 consumer group — lag 격리)	retention 24h — 이력 보관 책임 없음
Iceberg / MinIO	장기 이력 · 복구 원본 (envelope-as-is, append-only)	1일 파티션 drop = 보존 정책
Oracle 타깃	현재 상태 (PK upsert 먹등 apply)	—

3. 핵심 설계 판단

먹등 apply를 전제로 한 복구 단순화

전달 보장은 at-least-once로 두고, apply를 PK 기반 upsert(MERGE)로 통일해 **몇 번을 재적용해도 결과가 같게** 만들었다. 복구 재생의 경계 중복을 정밀 절단 없이 흡수하는 근거이며, PK 없는 테이블은 등록 단계에서 거부한다. 복구는 재발행 단일 경로 — recovery-job은 Iceberg를 읽어 복구 토픽에 재발행까지만 하고, apply는 라이브와 동일한 JDBC sink 설정이 담당해 **복구 결과와 라이브 적재가 달라질 수 없는 구조**다.

changelog 스키마를 실측 실험으로 확정

원안(평안화 스키마)이 기성 커넥터·SMT 조합으로 구현 불가함을 실험으로 증명하고 (스톡 SMT는 중첩 필드 승격 불가, 기존 변환 SMT는 before 이미지 소실 — 무손실 재조립 불변식 위반), **envelope 구조 그대로 저장 + backend가 테이블 사전 생성 + 1일 파티션**으로 설계를 개정했다. envelope 왕복 테스트(원본 → Iceberg 행 → 재조립 동등성)가 이 불변식의 회귀 방어선이다.

캡처 장애의 재현·진단·전환 — 유실 0

공유 dev Oracle에서 LogMiner redo_log_catalog 전략이 ORA-1371(complete dictionary not found) 재시도 루프에 빠져 커넥터는 **RUNNING**인데 **캡처가 멈추는** 장애를 재현했다. 디렉터리 아카이브 존재를 소스 디렉터리 조회로 확인해 전략 자체의 취약점으로 진단, online_catalog 로 전환(DDL 이벤트는 양 전략 모두 발행됨을 확인해 승인 워크플로 유지)했고, 소스 redo 재마이닝으로 밀린 9,673건을 전량 회수했다. 커넥터 삭제 후 재등록 시 잔존 offset으로 스냅샷이 건너뛰어지는 함정도 실측으로 발견해 등록 해제 시 offset 정리를 자동화했다.

캡처 계정 최소 권한 8개 도출

Debezium 문서의 보수적 권한 목록(20여 개)을 실 배선의 권한 실패를 재현하며 8개로 압축. `SELECT ANY DICTIONARY` 가 V\$ 뷰 개별 grant를 대체함을 확인하고, `DBMS_LOGMNR_D` (디렉터리 전략 전용)· `FLASHBACK ANY TABLE` (초기 스냅샷 전용 — flashback query 일관성)의 용도를 구분해 문서화했다. 사전 점검이 권한 누락 시 DBA용 GRANT 스크립트를 생성해 안내하고 배포를 차단한다.

운영 동작의 원칙화

jdbc-sink를 테이블별 커넥터로 분리해 타깃 이름 매핑과 테이블 단위 정지(DDL 거부 격리)를 가능하게 했고, recovery-sink는 apply 완료(consumer lag 소진)를 감지해 자동 정지한다("평시 정지" 원칙). 복구 트리거의 자동 재개 옵션으로 **"정지 → S3 복구 → 라이브 복귀(go-live)"**가 **한 번의 조작으로 완결**된다 — 경계 중복은 전부 역등이 흡수하므로 절단 시점 계산이 필요 없다.

4. 웹 콘솔 주요 기능

화면	기능
토폴로지	파이프라인 캔버스(React Flow) — 커넥터 상태 실시간, RUNNING 구간 흐름 애니메이션
CDC 등록 위저드	6단계: 소스 선택 → 디렉터리 조회(와일드카드 전개, PK 없는 테이블 선택 차단) → 타깃 → 컬럼 매핑(체크박스 제외· \${소스컬럼} 변환식·구문 검증) → 사전 점검(supp.log 승인 후 적용·권한 GRANT 안내) → 스냅샷 모드 선택(초기적재/현재부터) · 배포
테이블 모니터링	실측 지표 — 총 이벤트(offset)·이벤트/s·sink별 lag(AdminClient), 정지/재개/삭제(changelog 보존·삭제 선택), 상태 배지
DDL 이력	schema change topic 상시 수집(retention 만료 대비 DB 적재), 승인(타깃 이름 치환 실행)/거부(테이블 단위 apply 정지), 비전파성 DDL 자동 무시
이벤트	테이블별 운영 이력 — 등록·정지·DDL·복구·커넥터 장애 전이(원인 trace 포함) 타임라인
복구	changelog(S3) 현황(용량·보존 구간·파티션별) · SCN 지정 재발행 · go-live 자동 재개 · SRC/TGT 정합 검증(행수+체크섬)

5. 검증 결과

- ✓ 실 Oracle end-to-end: 초기 스냅샷(30,088행) → LogMiner 실시간 스트리밍 → 타깃 apply(lag 0) + changelog 커밋 까지 실패트릭 확인
- ✓ 캡처 장애(ORA-1371) 재현 → 전략 전환 → 밀린 9,673건 유실 0 회수
- ✓ envelope 왕복 동등성·changelog 사전 생성(실 카탈로그)·복구 재발행 스모크(73건) 등 자동화 테스트 72건
- ✓ 사전 점검 게이트 실행작: LogMiner 권한 누락 감지 → GRANT 안내 → 부여 후 통과

6. 기술 스택

Java 21 Spring Boot 3.5 MyBatis Debezium 3.6 (Oracle LogMiner · JDBC sink) Kafka 4.3 KRaft Apache Iceberg 1.11
MinIO(S3) PostgreSQL 15 React 19 TypeScript Vite Tailwind 4 · shadcn/ui React Flow · TanStack Table

데이터 경로는 기성 커넥터만 사용(자체 코드는 제어면·복구·UI에 한정 — apply 시맨틱 단일 경로 유지가 설계 원칙). 인프라는 베어메탈 스크립트로 먹등 구축(Iceberg Kafka Connect sink는 공식 프리빌드가 없어 소스 빌드). Iceberg 읽기는 Spark/Trino 없이 iceberg-data 라이브러리 단일 프로세스.

7. 남은 과제

복구 리허설 풀 사이클(타깃 훼손 → SCN 재발행 → 정합 검증 일치) 실행 검증 · 기동 중 테이블 추가 시 초기적재(incremental snapshot + Kafka signal) · 컬럼 리네임의 적재 반영(스톡 sink 한계 — 저장·검증까지 구현, 데이터 경로 정책 결정 대기) · 소스 대비 캡처 지연(초 단위) 지표